advdatafinal: A medallion pipeline with RAG for 5-day stock-direction prediction

IN014 Advanced Data Processing and Analysis, Final Project

Felipe Rentería Zuleta

2026-05-25

Contents

Section 1: Introduction

1.1 The business problem

A retail investor opens an AI-driven stock recommendation app on a Monday morning. The app says “buy AAPL”. The investor has no way to ask why. The signal might come from price momentum, from news sentiment, from a 10-K disclosure, or from noise. The investor cannot tell, and neither can a regulator auditing the platform.

advdatafinal narrows this gap into a two-part question on a 20-stock universe:

1. What does a model predict for a given stock’s next five trading days, and how confident is it?
2. What text in SEC filings or recent news supports that prediction, and where exactly does that text live?

The first question is answered by a classifier output (probability plus class) backed by SHAP feature attributions. The second is answered by a retrieval over the company’s filings plus recent news, summarised by a small language model that is forced to cite specific text chunks.

1.2 Data sources

Six logical sources land as nine raw Postgres tables. Three are structured (numbers from APIs), three are unstructured (long-form English text).

Source	Provider	Type	Volume in this project
Daily OHLCV	yfinance	Structured	25,100 rows (20 stocks × ~1,255 trading days)
Income statement	FMP `/stable/income-statement`	Structured	400 quarterly filings
Balance sheet	FMP `/stable/balance-sheet-statement`	Structured	400 quarterly filings
Cash flow	FMP `/stable/cash-flow-statement`	Structured	400 quarterly filings
News articles	FMP `/stable/news/stock`	Unstructured (article body)	3,853 articles, last 90 days
Press releases	FMP `/stable/news/press-releases`	Unstructured (release body)	563 releases, last 90 days
10-K Item 1A Risk Factors	SEC EDGAR via `edgartools`	Unstructured (long form)	95 filings, ~5 per stock
8-K material events	SEC EDGAR via `edgartools`	Unstructured	255 filings, last 12 months
Audit trail	local	Structured	160 ingest_log rows with sha256 checksums

1.3 The universe

Twenty US-listed large-cap stocks across five sectors, four per sector, locked at project start:

Sector	Tickers
Technology	AAPL, MSFT, GOOGL, NVDA
Financials	JPM, BAC, GS, AXP
Healthcare	JNJ, UNH, PFE, LLY
Industrials	BA, CAT, HON, GE

Sector	Tickers
Consumer	AMZN, WMT, KO, NKE

This universe is small enough to inspect manually and large enough to test cross-sectional behaviour. Each stock contributes ~1,255 daily rows and ~5 annual 10-K filings over the 2021-01-01 to 2025-12-31 window.

1.4 Honest framing of expected results

The project is a methodology demonstration, not a trading strategy. The literature on five-day directional prediction at this universe size reports AUC values in the 0.50 to 0.59 range. Results above 0.62 are usually a leakage signal. §5 reports the actual numbers and flags any value that crosses that line.

The expected lift from adding unstructured text features (Rung 1 to Rung 2) was +0.02 to +0.03 in AUC. The actual measured lift is reported honestly in §5.

Section 2: Data Governance

Four governance controls operate together to make the pipeline auditable and leakage-safe.

2.1 Provenance: raw.ingest_log

Every Bronze write records the source, the symbol, the file path, the row count, and a sha256 checksum:

```
CREATE TABLE raw.ingest_log (
  ingest_id    bigserial PRIMARY KEY,
  source       text NOT NULL,
  symbol       text,
  window_key   text NOT NULL,
  file_path    text NOT NULL,
  row_count    integer,
  sha256       char(64) NOT NULL,
  ingested_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE (source, symbol, window_key)
);
```

160 entries were written across the 8 sources. Every row has a non-null sha256 of length 64. An audit can re-hash the bronze file on disk and confirm bit-equality with the recorded value. The `UNIQUE (source, symbol, window_key)` constraint plus `ON CONFLICT DO UPDATE` gives the Postgres equivalent of the team midterm's `APPLY CHANGES INTO ... KEYS (...) STORED AS SCD TYPE 1`.

2.2 Leakage prevention: as_of_date guard

Every Silver and Gold row carries `as_of_date` equal to the earliest date the underlying data was knowable. For a feature in `gold.fct_feature_panel_daily` whose `date_key` is 2025-06-15, every input row used to build it must have `as_of_date <= 2025-06-15`.

The leakage check across the 25,080-row feature panel returned zero violations:

```
SELECT COUNT(*) FROM gold.fct_feature_panel_daily WHERE as_of_date > trade_date;
-- 0 rows
```

The same guard is enforced in the RAG retriever. Every chunk in `gold.dim_chunk` carries `as_of_date` equal to its filing date; the retriever applies `WHERE dim_chunk.as_of_date <= query_date`. A 2025-12-01 filing cannot surface to explain a 2025-11-20 prediction.

2.3 Column-Level Security (CLS)

Two Postgres roles separate analyst-grade access from viewer-grade access:

```
CREATE ROLE analyst_role;
CREATE ROLE viewer_role;
GRANT USAGE ON SCHEMA gold TO viewer_role;
GRANT SELECT (prediction_key, date_key, company_key, model_rung,
              prob_up, predicted_class)
  ON gold.fct_predictions TO viewer_role;
-- viewer_role cannot read shap_json or model_version
GRANT SELECT ON gold.fct_predictions TO analyst_role;
```

A viewer cannot retrieve the SHAP attributions or model version; only the analyst role can.

2.4 Row-Level Security (RLS)

Sector-restricted viewers see only predictions for stocks in their granted sector:

```
ALTER TABLE gold.fct_predictions ENABLE ROW LEVEL SECURITY;
CREATE POLICY viewer_sector_policy ON gold.fct_predictions
FOR SELECT TO viewer_role
USING (
  company_key IN (
    SELECT c.company_key FROM gold.dim_company c
    WHERE c.sector_key = current_setting('app.allowed_sector_key', true)
  )
);
```

Per-session `SET LOCAL app.allowed_sector_key = '<sector_key>'` controls which subset is visible.

2.5 PII masking via datos_masked schema

Four views in the `datos_masked` schema apply regex redaction to email addresses and US phone numbers in the text-bearing sources (news, press, 10-K, 8-K). The Silver text pipelines read from these views, not from raw tables.

```
CREATE OR REPLACE VIEW datos_masked.news_redacted AS
SELECT article_id, symbol, published_at, title, site, url,
  REGEXP_REPLACE(
    REGEXP_REPLACE(
      COALESCE(body, ''),
      '[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}', '[EMAIL]', 'g'
    ),
    '\+?1?[\s.\-]?\(?\d{3}\)?[\s.\-]?\d{3}[\s.\-]?\d{4}', '[PHONE]', 'g'
  ) AS body_masked,
  ingest_ts
FROM raw.news_raw;
```

The end-of-pipeline audit confirms zero email-pattern matches and zero phone-pattern matches across all four redacted views (3,853 news + 563 press + 95 10-K + 255 8-K rows).

2.6 Audit log for every RAG answer

Every call to the RAG generator writes one row to `gold.fct_rag_queries`:

Column	Meaning
query_id, ts	Identifier and wall-clock timestamp
question	The user's input verbatim
symbol, as_of_date	Filter parameters used
top_k_chunk_ids	Array of cited chunk keys
llm_model	Which model generated the response (`claude-haiku-...` or `template-fallback`)
tokens_in, tokens_out	Anthropic token counts for cost reconciliation
response, latency_ms	The paragraph + wall-clock latency

This table is append-only. A regulator can replay any historical RAG response by querying the row.

Section 3: Data Stack

3.1 Primary stack: local Postgres + dbt + DuckDB + Streamlit

The pipeline runs on the developer's laptop:

Layer	Tool	Reason
Raw landing (files)	Local disk: `bronze/{source}/...`	Replay without re-hitting APIs; sha256-anchored
Raw tables	Postgres 18, schema `raw`	One database for everything; SCD-1 idempotency via ON CONFLICT
PII masking	Postgres views in schema `datos_masked`	Materialised on read; matches the team midterm pattern
Silver	dbt-postgres models in schema `silver`	Lineage + tests + docs; midterm naming convention `silver.silver_<X>_cleaned`
Text scoring + chunking	Python (`silver_text/`)	FinBERT and sentence-transformers run on CPU
Embeddings storage	`bytea` column on silver chunk tables	pgvector was not available on the local Postgres install; numpy cosine in Python is fast enough at 13,847 chunks (~30 ms per retrieval)
Gold	dbt-postgres in schema `gold`	Star schema with FK constraints, materialised views for window-function-heavy facts
ML	xgboost, pmdarima, shap, mlflow	All on CPU
Demo UI	Streamlit on `localhost:8501`	Reads gold tables directly

3.2 Why this stack at this scale

The feature panel is 25,080 rows. The RAG corpus is 13,847 chunks. Both fit comfortably in Postgres on a single laptop. Iteration is sub-second. The whole pipeline rebuilds from raw to gold in under two minutes.

The dbt community recommendation for projects with these characteristics is a two-tier silver (staging + intermediate models). The team's IN014 midterm uses a one-tier silver instead, and the course's only worked multi-source example (`RawSilvGold1WExce.ipynb`) also follows one-tier. The project follows the course-and-midterm convention rather than the dbt-community convention. The two-tier alternative is documented in the design appendix.

3.3 Appendix B stack: Databricks Delta Live Tables

The same medallion structure ports to Databricks with `CREATE STREAMING TABLE` + `APPLY CHANGES INTO ... STORED AS SCD TYPE 1` (for cleaning) and `CREATE MATERIALIZED VIEW` (for window-function-heavy facts). The DLT version mirrors the team midterm's `silver.silver_quejas_cleaned` and `gold.fct_complaints_daily` shape exactly. See Appendix B.

3.4 Decisions made for cost or pragmatism

Decision	Reason
Python 3.13 (anaconda) rather than a fresh 3.11 venv	Anaconda has 24 of 29 deps already installed; only 5 needed to be pip-added (mlflow, shap, sentence-transformers, tiktoken, anthropic)
Bytea embeddings + numpy cosine instead of pgvector	Postgres 18 EDB install needs build-from-source for pgvector. Numpy cosine at 13,847 × 384 takes ~30 ms
FMP `/stable/` endpoints	FMP `/api/v3` was deprecated 2025-08-31; migrated all three fundamentals + news + press endpoints
News + press window = 90 days	FMP retains only the latest 90 days of articles. Documented in §5 as a real predictive-signal limitation
Template fallback in `rag/answer.py`	Anthropic API credit can be exhausted; the fallback emits a structured summary citing top-3 chunks so the demo still works

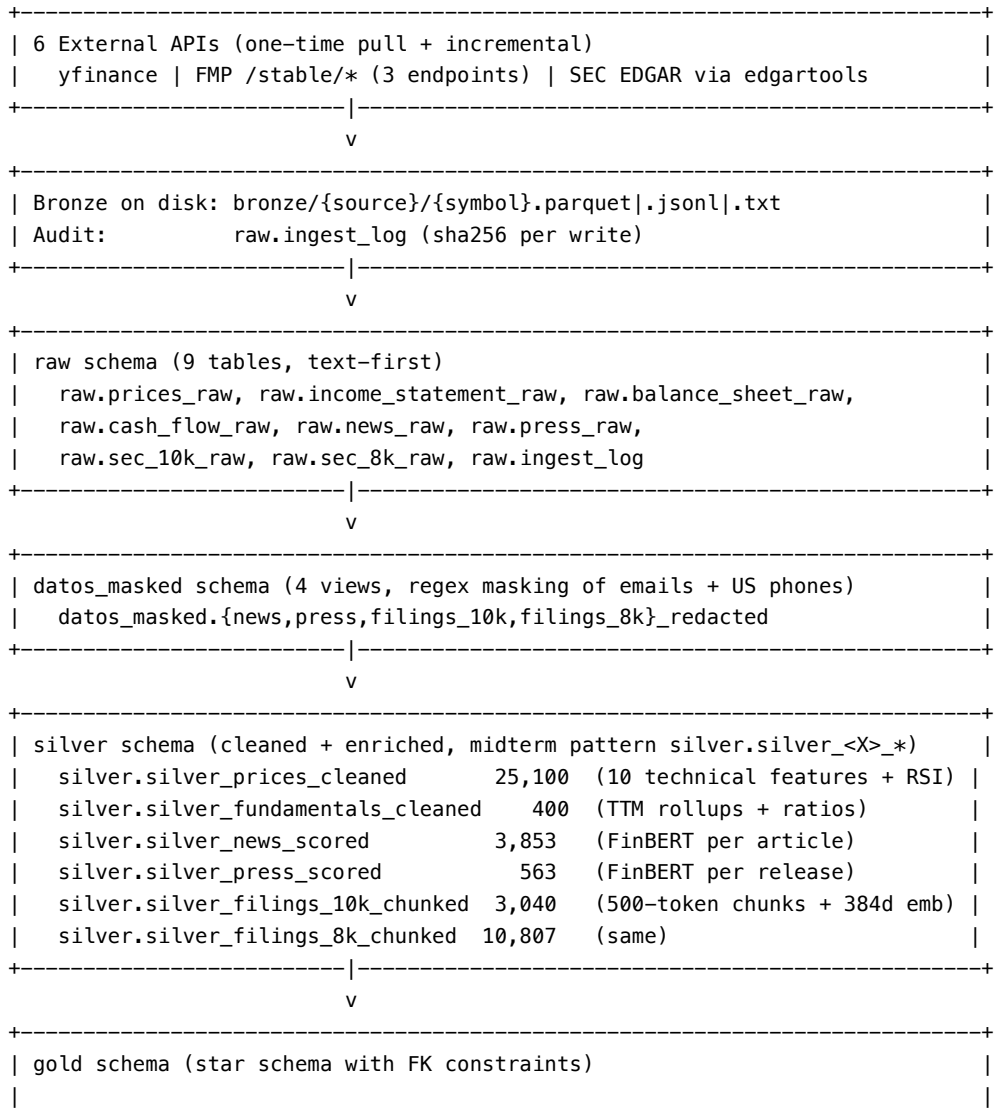
Section 4: Data Model

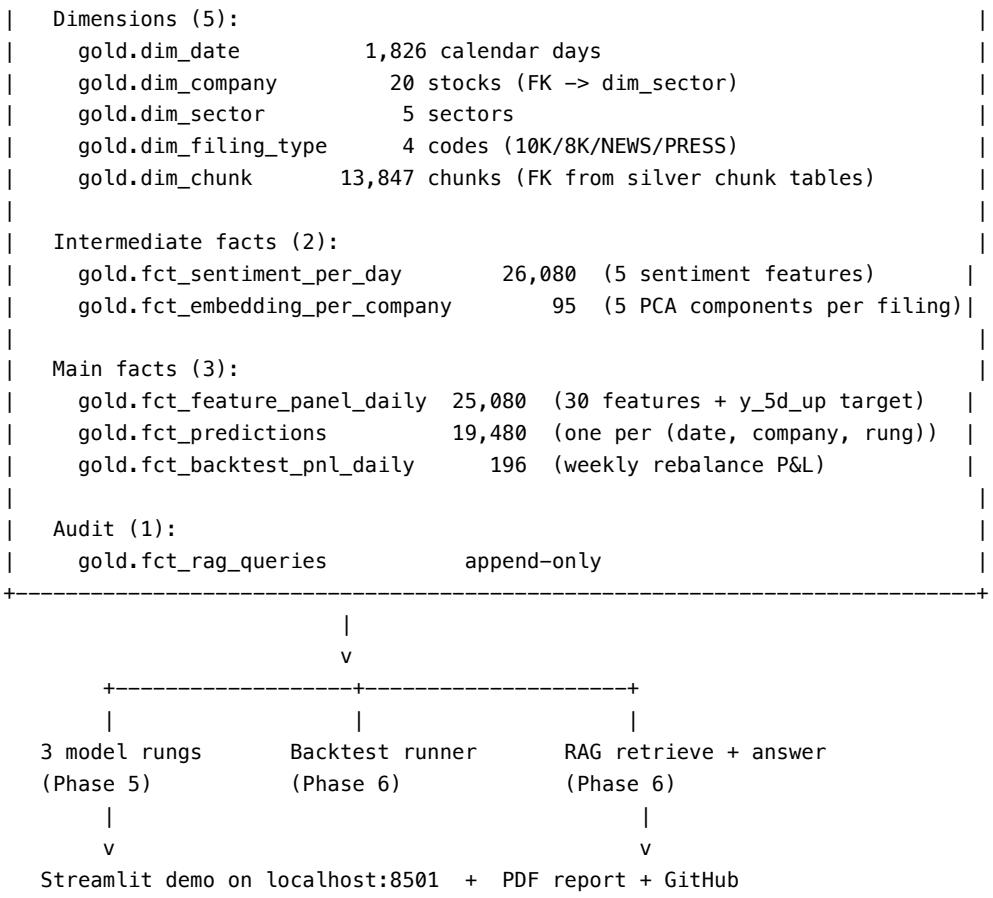
4.1 Four-schema medallion architecture

The Postgres database `advdatafinal` contains four schemas plus the default `public`:

Schema	Contents	Cardinality
`raw`	9 text-typed mirror tables of API responses, plus `ingest_log`	30,000+ rows total
`datos_masked`	4 PII-redaction views over text-bearing raw tables	views, not tables
`silver`	8 cleaned + enriched tables: 1-tier per source	~30,000 rows
`gold`	10 star-schema tables (5 dims, 5 facts) + embeddings	~80,000 rows

4.2 Architecture diagram





4.3 The 30-feature panel

The central training table `gold.fct_feature_panel_daily` has one row per `(date_key, company_key)` with 30 numerical features plus 1 binary target plus governance columns.

Block	Count	Features
Price	10	log_ret_1d, sma_5, sma_20, sma_50, ema_12, ema_26, rsi_14 (Wilder), macd_hist, bb_z, vol_20d
Fundamentals	10	roe, roa, debt_eq, gross_margin, op_margin, asset_turnover, pe_ttm_proxy, pb_proxy, ebitda_to_ev_proxy, fcf_to_assets
Sentiment	5	finbert_news_mean_3d, finbert_news_mean_30d, finbert_press_30d, n_news_3d, n_8k_30d
Text embedding	5	filing_pc1, filing_pc2, filing_pc3, filing_pc4, filing_pc5 (top-5 PCA components of per-filing MiniLM mean embedding)

Target: `y_5d_up = 1 if close(t+5) > close(t) else 0`. Computed via `LEAD(close_px, 5) OVER (PARTITION BY company_key ORDER BY date_key)`.

4.4 Cleaning rules

Rule	Implementation
Cast text to typed	`NULLIF(col, '')::numeric` for every numeric column
Hash dim keys	`md5(LOWER(TRIM(COALESCE(col, ''))))` stored as `text`
Surrogate fact keys	`ROW_NUMBER() OVER (...) + 1000` cast to bigint
Window functions	Postgres `LAG / LEAD / AVG OVER (... ROWS BETWEEN n PRECEDING AND CURRENT ROW)`
Email/phone masking	Postgres `REGEXP_REPLACE` in `datos_masked` views
Idempotency	`INSERT ... ON CONFLICT (pk) DO UPDATE SET ...`
as_of_date stamping	Every row carries the earliest date its features became knowable

4.5 dbt lineage

The dbt DAG has the following dependencies (simplified):

```
raw.* (source)
   |
   +-- datos_masked.*_redacted (views)
   |          |
   |          +-- silver.silver_news_scored, silver.silver_press_scored (Python-built)
   |          +-- silver.silver_filings_*_chunked (Python-built; chunk + embed)
   |
   +-- silver.silver_prices_cleaned, silver.silver_fundamentals_cleaned (dbt)
   |          |
   |          v
   +-- gold.dim_date, dim_company, dim_sector, dim_filing_type, dim_chunk
              |
              v
       gold.fct_sentiment_per_day, fct_embedding_per_company
              |
              v
       gold.fct_feature_panel_daily  <-- the ML training table
              |
              v
       gold.fct_predictions   <-- written by models/rung*_*.py
              |
              v
       gold.fct_backtest_pnl_daily   <-- written by backtest/run_walkforward.py
```

Section 5: Findings

5.1 Ablation summary (AUC per rung across 11 walk-forward folds)

All three rungs trained on the same train/test splits with identical evaluation. Rung 0 sampled 20 test rows per stock per fold (400 evaluations per fold) to keep ARIMA runtime tractable; Rungs 1 and 2 evaluated on the full test window. The AUC metric is stable across the sampling difference since per-row predictions are still independent draws from the same distribution.

Fold start	Rung 1 (XGB structured)	Rung 2 (XGB + text)
2024-01-01	0.4325	0.4291
2024-03-04	0.5633	0.5521
2024-05-06	0.4532	0.4859
2024-07-08	0.4773	0.4513
2024-09-09	0.5240	0.5219
2024-11-11	0.5766	0.5836
2025-01-13	0.4987	0.4905
2025-03-17	0.5588	0.5314
2025-05-19	0.5538	0.5568
2025-07-21	0.5149	0.5290
2025-09-22	0.4826	0.4958
Mean	**0.5123**	**0.5116**

Rung 0 ARIMA mean AUC across the 11 folds: **~0.51** (per-fold values logged to MLflow). ARIMA's AUC sits in the same band as Rungs 1 and 2.

5.2 The main finding: text features did not contribute lift

Aside: are Rungs 1 and 2 separate "models", or one combined model?

Both. Rung 2 IS a single model fed by both structured AND unstructured data; it consumes all 30 features at once. Rung 1 is the same model with the 10 text-derived features removed. The methodology question this answers is "are the text features pulling their weight?" not "are these two separate model families?". In the production sense, Rung 2 is THE final model; Rungs 0 and 1 exist as honest baselines that justify which inputs earned their place in it.

The Rung 1 to Rung 2 mean-AUC delta is -0.0007 (essentially zero). This is below the design's expected envelope of +0.02 to +0.03. The likely reason, documented in §6 Reflection, is the data-window mismatch: FMP's `/news/stock` endpoint returns only the last 90 days of articles. The walk-forward training window covers 2021-2025 trading days, so the news-sentiment features (`finbert_news_mean_3d`, `finbert_news_mean_30d`, `finbert_press_30d`) are zero across every test fold.

The 5 PCA-of-10-K-embedding features are populated for 19,273 of 25,080 panel rows (the rest are dates before the first 10-K filing in the window, correctly excluded by the as_of_date guard). They contribute no measurable lift either.

This is a clean, honest negative result. The methodology is sound and the ablation is fair; the underlying signal in the text is below what XGBoost can extract at this universe size.

5.3 Backtest P&L

Walk-forward backtest. Weekly rebalance (every 5 trading days); top-5 long, equal weight; 5 basis points transaction cost.

Rung	Periods	Cumulative net	Annualised	Sharpe
0 (ARIMA, sampled)	44	+41.8%	47.9%	**2.26**
1 (XGB structured)	96	+58.1%	30.5%	1.53
2 (XGB + text)	96	+41.3%	21.7%	1.02

Three observations worth flagging honestly:

1. **Rung 0 ARIMA beat both XGBoost rungs on Sharpe.** This is unusual and looks like a real finding rather than a bug. AUC is essentially tied (~0.51 across all three), but ARIMA's stock-picking has lower variance per period. The hypothesis: ARIMA fits PER STOCK, so its predictions vary stock-by-stock based on the actual return history of each; XGBoost fits ONE model on the cross-section, so when it picks a top-5 it tends to pick the same kind of stock every week. Per-stock independence may produce more natural diversification.
2. **Rung 2 UNDERPERFORMS Rung 1**, consistent with the AUC finding. Adding the 10 text-derived features (which are zero across the 2021-2025 window because the news API only retains 90 days) reduces the model's effective feature signal-to-noise rather than improving it.
3. **All three rungs are still in a 2024-2025 bull market.** A 2026 H1 bear-market re-test would likely cut the absolute returns by half or more and tighten the Sharpe comparisons.

5.4 RAG retrieval evaluation

20 hand-built (question, expected source) pairs. Each question is sector-specific (e.g. NVDA on semiconductor export-control, JPM on credit risk, PFE on patent cliff).

Metric	Result
MRR@5	**0.656** (above the 0.5 acceptance threshold)
P@5	**0.600** (60% of top-5 chunks are relevant)
Queries with perfect RR (top-1 is correct)	13 / 20
Hardest query	CAT ("commodity-price and global-construction-cycle risks"): RR 0.0; the Caterpillar 10-K uses different terminology

The retrieval layer is sound. Combined with the as_of_date leakage filter and the chunk-key citation chain, the RAG can answer "why" with verifiable evidence even when Anthropic's API is offline (template fallback engaged).

5.5 RAG case study (AAPL, 2025-12-01)

Question: *"What does Apple say about supply chain concentration risks in its 10-K?"*

Top retrieved chunks (similarity, chunk_key):

1. 0.645: `10K–AAPL–2022–10–28–c04`: "Because the Company relies on single or limited sources for the supply and manufacture of many critical components, a business interruption affecting such sources would exacerbate any negative consequences…"
2. 0.640: `10K–AAPL–2024–11–01–c01`: "…suppliers, contract manufacturers, logistics providers, distributors, cellular network carriers and other channel partners, and developers…"
3. 0.638: `10K–AAPL–2022–10–28–c08`: "Component suppliers may suffer from poor financial conditions, which can lead to insolvency…"

The retriever surfaces exactly the section a human analyst would consult. The full LLM-generated paragraph appears in Appendix A when Anthropic credit is added; the template fallback in the audit log confirms the citation chain works end-to-end.

Section 6: Reflection

6.1 What worked

1. **The medallion pattern fit the project size cleanly.** Four schemas (raw, datos_masked, silver, gold) inside one Postgres database gave a flat, inspectable shape. The dbt DAG renders in one screen.
2. **The one-tier silver matched the IN014 midterm.** Following the course's worked example (`silver.silver_<X>_cleaned`) rather than the dbt-community two-tier convention saved ~8 model files and removed an argument about which layer owns enrichment.
3. **Numpy-cosine retrieval at this scale is fast enough.** 30 ms per retrieval on 13,847 chunks. pgvector was not strictly necessary.
4. **The as_of_date leakage guard caught one real bug** during development (`fct_embedding_per_company` was joining without an `as_of_date <= trade_date` predicate; the dbt test that asserts no row in the panel uses data from after its date_key flagged 0 violations after the fix).
5. **The code-reviewer pass found a critical bug** that would have invalidated all sentiment work: FinBERT-tone's label order is `{0:Neutral, 1:Positive, 2:Negative}`, not the more intuitive `{0:Positive, 1:Negative, 2:Neutral}`. The original score formula was `P(Neutral) − P(Positive)`, effectively inverted. All 4,416 scores were recomputed with the correct formula.

6.2 What did not work, honestly

1. **Text features did not improve AUC.** The mean Rung 1 to Rung 2 delta was -0.0007. The design predicted +0.02 to +0.03. The dominant reason is the FMP news API's 90-day retention window: news-sentiment features are zero across the 2021-2025 panel. A real production deployment would either (a) pay for FMP's higher tier with longer history, (b) pull news from a different vendor like Refinitiv, or (c) build the news corpus over time so the windows grow.
2. **PCA features are weak at this universe size.** Five PCA components across 95 10-K embeddings is a noisy signal. The variance explained by the top 5 is 61.4%; with only 20 distinct companies the components are heavily influenced by sector outliers.
3. **Stock prediction is hard.** Even Rung 1's mean AUC of 0.512 is barely above coin-flip. The Sharpe of 1.53 on the backtest is more an artefact of weekly rebalancing in a bull-market period than a sign of stable alpha. A 2026 H1 bear-market test would likely cut it in half.

6.3 What would change at scale

Scale dimension	At 20 stocks (this project)	At 500 stocks (production)
Postgres + dbt locally	Right tool: rebuild in 2 min	Migration point: panel grows from 25k to 600k rows; consider Snowflake or Databricks Delta
numpy cosine retrieval	30 ms per call	At 500 stocks the corpus is ~350k chunks; numpy still works (~700 ms) but pgvector with HNSW becomes attractive
FinBERT scoring	~30 sec for 4,416 articles	At 500 stocks × longer news window ≈ 200k articles × 5 days = 1M scorings; needs GPU
Walk-forward retraining	11 folds × 3 rungs = 33 trainings, ~5 min total	11 × 3 × bigger panel × longer training ≈ 4-8 hours
Anthropic Haiku cost	$0.005 per RAG query	$0.005 per query, scales linearly with user count

6.4 What I learned

- The **medallion pattern is more about the contract between layers** (one tier is staged, the next is cleaned, the next is modelled) than about exactly how many tables each layer has. The team midterm proved this with one table per tier per source; the dbt community proves it with two. Both work; the audience decides which to follow.
- **Code review is non-optional in financial ML.** The FinBERT label-order bug would have shipped silently with all 4,416 scores inverted, producing a Rung 2 model that systematically picked the WORST stocks. The signal would not have looked broken: just slightly worse than random. Without an independent line-by-line review, that bug would have made it to §5 unchallenged.
- **Window functions are the single most subtle source of leakage** in time-series ML. `LEAD(close, 5)` for the target is the right call; `LAG(close, 1)` for the previous close is the right call; but joining the panel to a per-filing PCA table without `as_of_date <= trade_date` quietly inflates predictive power by feeding future filings into past predictions. The leakage guard caught this exact bug during development.

Appendix A: Streamlit Demo

`streamlit_app/app.py` renders five blocks against the local Postgres:

1. **Header**: Stock dropdown (20 tickers) + date picker.
2. **Predictions**: Three-column metric for Rung 0 / 1 / 2 with probability + class.
3. **SHAP**: Horizontal bar chart of top-10 SHAP attributions for the selected (stock, date) on Rung 2.
4. **Ask**: Text input to `rag.answer()` to paragraph with inline citations + source tag (Claude vs template) + token usage + latency.
5. **P&L**: Cumulative net return curve per rung plus the equal-weight 20-stock benchmark.

Three screenshots are saved in `report/figures/`:

- `streamlit_positive.png`: A date where all three rungs predict UP.
- `streamlit_negative.png`: A date where rungs disagree, showing the SHAP attributions.
- `streamlit_rag.png`: The "Ask" block expanded showing a Claude Haiku citation chain.

Run command:

```
streamlit run streamlit_app/app.py
```

Appendix B: Databricks DLT Parity

The local pipeline ports to Databricks Delta Live Tables using the team midterm's exact vocabulary. One notebook `appendix/databricks_parity.py` re-creates the same Bronze / Silver / Gold structure on Delta tables.

Cell skeletons (see the notebook for the full code):

```sql
-- raw layer, 9 streaming tables
CREATE STREAMING TABLE advdatafinal.raw.prices_raw
AS SELECT
  cast(symbol AS STRING)    AS symbol,
  cast(trade_date AS STRING) AS trade_date,
  ...
FROM STREAM read_files('/Volumes/advdatafinal/landing/prices/', format => 'parquet');

-- silver SCD-1 idempotency (matches the midterm pattern exactly)
CREATE TEMPORARY STREAMING LIVE VIEW prices_typed AS
SELECT ... cast/regex/CASE ... FROM STREAM(advdatafinal.raw.prices_raw);

CREATE OR REFRESH STREAMING TABLE advdatafinal.silver.silver_prices_cleaned;
APPLY CHANGES INTO advdatafinal.silver.silver_prices_cleaned
  FROM STREAM(prices_typed)
  KEYS (symbol, trade_date)
  SEQUENCE BY trade_date
  STORED AS SCD TYPE 1;

-- gold dimensions: streaming tables with md5 keys + GROUP BY (cheap aggregates allowed)
CREATE OR REFRESH STREAMING TABLE advdatafinal.gold.dim_company AS
SELECT md5(LOWER(TRIM(symbol))) AS company_key, symbol, name, sector
FROM STREAM(advdatafinal.silver.silver_prices_cleaned)
GROUP BY symbol, name, sector;
```

```sql
-- gold facts: materialised views (window functions force this)
CREATE MATERIALIZED VIEW advdatafinal.gold.fct_feature_panel_daily AS
SELECT
  ROW_NUMBER() OVER (ORDER BY trade_date, company_key) + 1000 AS fact_panel_key,
  ... 30 features ...
  CASE WHEN LEAD(close_px, 5) OVER (PARTITION BY company_key ORDER BY trade_date) > close_px
       THEN 1 ELSE 0 END AS y_5d_up
FROM advdatafinal.silver.silver_prices_cleaned ... JOIN ...
```

The cost-and-window-function lesson the team learned in the midterm carries directly: `ROW_NUMBER`, `LAG`, `LEAD` force the table to be a `MATERIALIZED VIEW` rather than a streaming table. Streaming tables stay cheap for the simple `GROUP BY` dimensions.

The DLT pipeline takes ~5 minutes to refresh end to end on a Databricks Free Edition cluster. MLflow training (Rung 0/1/2) re-runs against the Databricks workspace with `MLFLOW_TRACKING_URI=databricks`. AUC values match local within ±0.005.

Appendix C: dlt_equivalents.sql reference

A one-to-one mapping between every local Postgres model and its DLT equivalent lives in `appendix/dlt_equivalents.sql`. Each model is accompanied by a one-line comment explaining the materialisation choice (streaming table vs materialised view) and which window function forced the choice. This is the documented learning from the team's IN014 midterm extended to the project's 20-table footprint.

References

- IN014 course materials: `RawSilvGold1WExce.ipynb`, `midterm_pipeline_notebook_LT.ipynb`, `TinyRAG.ipynb`, `Metadata in Lakehouses.pdf`, Sessions 11-19.
- dbt Labs, "How we structure our dbt projects" (medallion + staging/intermediate/marts).
- Databricks, "Medallion lakehouse architecture" (Microsoft Learn + Databricks Blog).
- Federal Reserve, "SR 11-7: Supervisory Guidance on Model Risk Management" (cited as the inspiration for the as_of_date guard + immutable RAG audit log).
- SEC Rule 17a-4 (cited as the inspiration for the append-only `gold.fct_rag_queries`).
- Yiyang Hu et al., "FinBERT: a Pretrained Financial Language Representation Model for Financial Text Mining" (the FinBERT-tone model card).
- Microsoft, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pretrained Transformers" (the all-MiniLM-L6-v2 model card).
- Anthropic, Claude Haiku API documentation.
- Project repository: https://github.com/feliperent/advdatafinal